CS561 : Concurrency-aware Algorithm Design for ZNS SSDs

Anastas Kanaris
Boston University

Bruce Casillas
Boston University

Ross Mikulskis
Boston University

ABSTRACT

Modern storage devices like ZNS SSDs offer high throughput and stable performance by giving the host more control over data placement and management. However, applications must design algorithms that take advantage of the device's internal parallelism and achieve maximum bandwidth. While recent research has led to the development of a novel parametric I/O (PIO) model for device characterization, this was designed for conventional block interfaces. To our knowledge, no similar research has been done into the ZNS interface, limiting their adoption into modern applications.

In this paper, we extend the previous research in PIO to the ZNS interface. We use the state-of-the-art SSD simulator ConfZNS++, which allows for fine-grained control over the configuration of SSD. This allows us to simulate the effects that different ZNS I/O management designs have. In performing these experiments, our aim is to provide an understanding of how to design systems for the ZNS interface to maximize throughput. We publish our findings in a public github repository (https://github.com/rkulskis/cs561-zns).

1 INTRODUCTION

1.1 Motivation

There has been a shift in recent years whereby processing power has switched from scaling vertically to scaling horizontally. Since the rate at which the number of transistors on a microchip doubles has dropped below the Moore's Law threshold, modern hardware has increasingly shifted to multi-core and multi-threaded architectures [5]. To fully leverage the capabilities of these hardware advancements, it is essential to design concurrency-aware algorithms. When done correctly, these enable parallelism, allowing multiple operations to be executed simultaneously, which leads to a more efficient use of CPU resources. They also help maximize CPU throughput by minimizing bottlenecks and coordinating access to shared structures. This in turn influences design decisions, as knowing how to use these technologies optimally allows engineers to make informed decisions about which is best suited for their application needs. Without this awareness, developers risk underutilizing hardware potential, leading to suboptimal performance and scalability.

One such area of hardware advancement has been in storage. The development of solid-state drives (SSDs) have allowed for them to become the de facto standard for storing and processing data at high speeds [4]. Traditional flash based SSDs maintain the popular block interface design that serves to abstract away hard drive media characteristics and simplify host software. However, the performance and operational costs of supporting the block interface are growing prohibitively [6]. This is due to a mismatch between the operations allowed and the nature of the underlying flash media.

As a byproduct of this, researchers introduced a new interface standard for flash-based SSDs known as ZNS [2]. This interface comes with a complexity cost, but it allows for the SSD to perfectly align the data with its media, providing significant efficiency benefits.

Despite this, the usage of ZNS SSDs in modern applications is still relatively infrequent. This is partially to do with the aforementioned complexity cost, but also because of the relative unknown that the ZNS interface represents. To get the most out of these new storage devices, it is imperative for modern applications to be able to design algorithms that take advantage of their internal parallelism.

For SSDs using the traditional block interface, research has already been done in this regard. The Parametric I/O Model (PIO) captures contemporary devices by parameterizing read/write asymmetry (α) and access concurrency (k). PIO enables device-specific decisions at algorithm design time, rather than as an optimization during deployment and testing, thus ensuring optimal algorithm design by taking into account the properties of each device [10].

1.2 Problem Statement

The same research has not been achieved for SSDs using the ZNS Interface. The methodology PIO uses can not immediately be translated to use in ZNS SSDs, as it assumes a block interface, which provides inherent data management abstractions. In contrast, ZNS SSDs shift this responsibility to the host, meaning that similar tests must be aware of ZNS requirements like sequential writes. Additionally, the ZNS interface effectively limits the number of zones that can be written in parallel, thus binding the parallelism.

The question we set out to answer is as follows:

How can we quantify read/write asymmetry and concurrency in SSDs using the ZNS Interface?

Answering this question involves using the state-of-the-art ConfZNS++ simulator, a novel SSD emulator which extends the flash emulator FEMU to test various SSD configurations [4, 9]. These developments have made it possible to accomplish the research needed to attempt to solve this problem.

1.3 Contributions

Our contributions can be summarized with the following. We explore the many different parameters of data management that modern ZNS SSDs have exposed to host software. We extend the PIO model to ZNS SSDs using the ConfZNS++ simulator, allowing us to adjust these parameters to test a variety of different configurations [4]. In doing this, we quantify the effective read/write asymmetry and internal concurrency of ZNS SSDs.

The format of the rest of this paper is as follows. **Section 2** provides a brief overview of SSD functionality and the advancements that have been made in recent years. **Section 3** looks at ConfZNS++ in more detail and provides an overview of our experimental process. **Section 4** covers the results of our experiments. Finally, **Section 5** provides our closing thoughts.

2 BACKGROUND

Figure 1 details the general hierarchy of how data is stored as it moves further away from the data system application. There is an inverse relationship between speed and size as you traverse down the pyramid. Lower levels on the pyramid have longer data access times, as they are stored further away from the application itself. However, they still have their place because they can store more data at a lower cost compared to upper levels.

Historically, traditional magnetic hard disk drives (HDD) made up the level immediately below main memory and above tape. In recent years, it has been shown that Flash SSDs strike a better balance between speed, size and cost when compared to HDD's. Today, SSDs can deliver microsecond access latencies, millions of I/O operations per second, and gigabytes of bandwidth per second [4]. As a result they have merged as a viable mainstream alternative, used in all aspects of the data management stack [8].

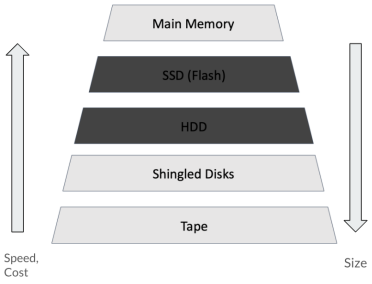

Figure 1: Storage Memory Hierarchy

2.1 General SSD Architecture

SSDs are made using NAND cells, a standard semiconductor non-volatile form of flash memory that can be programmed to store data [1]. An SSD contains multiple NAND packages organized into dies, planes, blocks, and pages. Figure 2 provides a visual depiction of this.

SSDs contain a few inherent properties. Reads occur at page level. Writes occur at page level, and updates are out-of-place, meaning that they are treated as new writes, and older ones are eventually erased over time. Erasure occurs at the block level. Finally, SSDs utilize wear-leveling, a technique that evenly distributes write and erase cycles across all NAND flash memory cells to prevent premature wear-out and extend the device's lifespan [3].

2.2 Black Box SSDs

Conventional NAND SSDs (CNS) typically have a single open block, within which a free page is programmed with the host's data. The host issues read and write commands to the SSD in terms of logical block addresses (LBAs), treating the device as a black box that abstracts all method implementation logic. The SSD is then responsible for handling how to optimally place data, out-of-place updates, garbage collection and wear leveling.

Traditional SSDs cannot be overwritten directly because they follow the erase-before-write principle. Erase operations in SSDs occur in terms of erase blocks (typically tens to hundreds of MBs

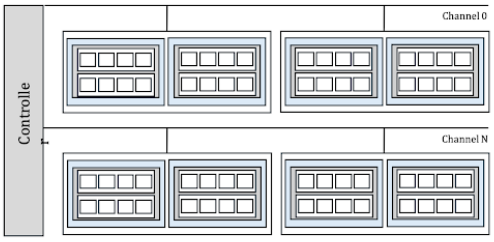

Figure 2: SSD Organization Levels

in size), whereas the write operations occur in terms of pages (typically 4KB, 16KB, etc. in size). This mismatch in erase and write granularities necessitates significant management as the host issuing an overwrite for the same LBA does not result in the SSD writing data to the same physical address internally. As a result, write amplification is a common drawback. Additionally, garbage collection can interfere with the host I/O operations, as well as increase the wear on the SSD. These drawbacks are referred to as the Block Interface Tax [2].

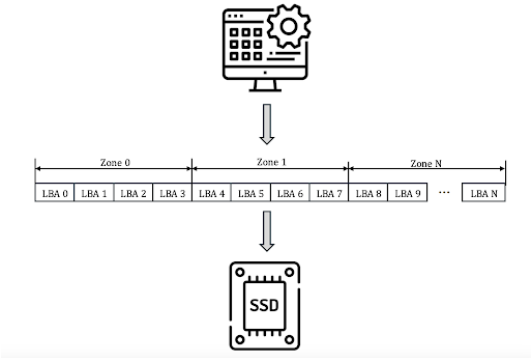

Figure 3: ZNS Interface

2.3 ZNS SSDs

To circumvent this Block Interface Tax, researchers introduced the NVMe Zoned Namespace Command Set specification, or ZNS for short, as a new interface standard for flash-based SSDs [2]. Figure 3 demonstrates the fundamental idea of the interface. Instead of a single array of logical blocks, a ZNS SSD groups logical blocks into zones that support random reads and sequential writes. This shifts the responsibility of data management from the hardware to the host. Although this incurs a complexity cost on the host software design, it allows the SSD to perfectly align the data with its media, providing significant efficiency benefits. For instance, this change in design reduces the reliance on internal garbage collection and over-provisioning, issues that have affected conventional block-interface SSDs [10].

This also shows why the application design is critical when working with the ZNS interface. To properly maximize the device's internal parallelism, developers need to carefully manage data placement while taking into account the sequential write constraint imposed by the zones.

2.3.1 ZNS vs Block Difference. The main difference in the two interfaces is summarized in figure 4. In short, Traditional SSDs rely on a block interface that abstracts the underlying flash memory characteristics, leading to complications such as write amplification. In contrast, ZNS SSDs expose the physical organization of the media by dividing the storage space into zones, each of which requires sequential writes and must be reset before reuse [2].

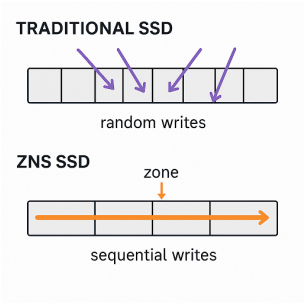

Figure 4: ZNS Interface vs Traditional Interface

2.4 Other Relevant Research

While research into ZNS SSD's is still new, research into the impact of concurrency on storage devices is not. In 2021, the Data-intensive Systems and Computing (DiSC) lab at Boston University published research about the development of a parametric I/O (PIO) model for characterizing conventional storage devices that accounts for read/write asymmetry (α) and the level of access concurrency (k) [10]. However, this model was developed with block-interface SSDs in mind and does not directly apply to ZNS devices. The absence of a similar study for the ZNS interface presents an opportunity to develop design strategies that leverage the unique characteristics of these devices for improved performance [7].

Adapting the PIO model for the ZNS interface introduces several important design considerations. One key factor is zone mapping, as ZNS SSDs can be configured with different architectures, ranging from small zones mapped to a single parallel unit to large zones that span multiple units. Another crucial aspect is host-managed data placement, which shifts data management responsibilities from the SSD firmware to the host, allowing for more device-specific optimizations at the algorithm design stage. Finally, balancing constraints and throughput requires careful coordination of I/O operations, as zones must be written sequentially and only a limited number can be active at a time. The extended PIO model provides a framework to assess these trade-offs and develop algorithms that maximize concurrency and bandwidth.

3 ARCHITECTURE

3.1 ConfZNS++

To test the impact of concurrency and asymmetry in ZNS SSD's, we used the ConfZNS++ emulator. Developed by the Storage and Network (StoNet) Research team at Vrije Universiteit Amsterdam, ConfZNS++ allows for users to simulate the characteristics of different SSD configurations in order to test the impact of a variety of I/O management strategies on them [4]. This level of tuning allowed is made possible because ConfZNS++ is built on top of the FEMU emulator, which effectively creates a virtual machine with a disk image using QEMU so that when compiled and run simulates having an attached disk drive with the specified configurations [9]. From there, one can run tests using flexible I/O tester (FIO) commands to evaluate the SSD's performance.

3.2 Experimental Setup

Our experimental setup went as follows. First, as ConfZNS++ is currently only supported in certain Linux environments, we developed our repository inside a shared cluster running on a Beelink 16GB Node using a Ubuntu Linux 6.5 kernel. We then installed and compiled the ConfZNS++ artifact, as well as the FEMU emulator. For the virtual environment, we ended up using the default FEMU image that is publicly made available, but it would also be possible to create your own image with a more advanced kernel. Once inside the Virtual Machine, we would run the various FIO tests, and send back the findings to our host machine.

We further streamlined this process by consolidating the code into a simple script that runs from our host machine.

3.3 Testing Design

Our infrastructure programmatically tests the different zone configurations of channels per zone, number of threads, and I/O request sizes. On the host side we modify the number of threads strided across zones. The search matrix is shown in Table 1 where the search space is the cartesian product of the 3 columns.

Table 1: Testing input matrix

(chans/zone,ways/zone)	I/O sizes	I/O type
SU=(1,1)	4KB	Read
MU4=(4,1)	128KB	Write
MU8=(8,1)		
FU=(8,2)		

3.3.1 ZNS Configurations. Essentially, there are four main ConfZNS++ configurations being checked: SU, meaning a 1-to-1 mapping of channels to zone, MU^4, meaning a 4-to-1 mapping, MU^8, meaning an 8-to-1 mapping, and FU, meaning the same mapping as MU^8 but with a 2-to-1 mapping of ways to zone.

3.3.2 FIO Configurations. For each ZNS configuration, we primarily tested by modifying two different FIO command arguments. The first parameter, "**–numjobs**" represents the number of threads spawned by the test. The idea behind this is that by increasing the number of jobs, we can simulate the number of concurrent accesses

to our disk. If each write goes to a different zone, this provides a way of testing parallelism across zones.

The other parameter modified was "**–iodepth**". This parameter represents the number of I/O requests issued to the OS at any given time. The idea behind modifying this parameter is that doing so might increase the level of parallelism and queuing within each job.

3.4 Benchmark Explanations

As a reminder, we are attempting to characterize the proper access concurrency k and read/write asymmetry α.

4 RESULTS

In this section we attempt to summarize the key findings of our experiments. As mentioned above, all experiments were run remotely on a Beelink 16GB shared cluster, with a Ubuntu Linux distribution running Linux v6.5.

4.1 Inter-Zone Parallelism

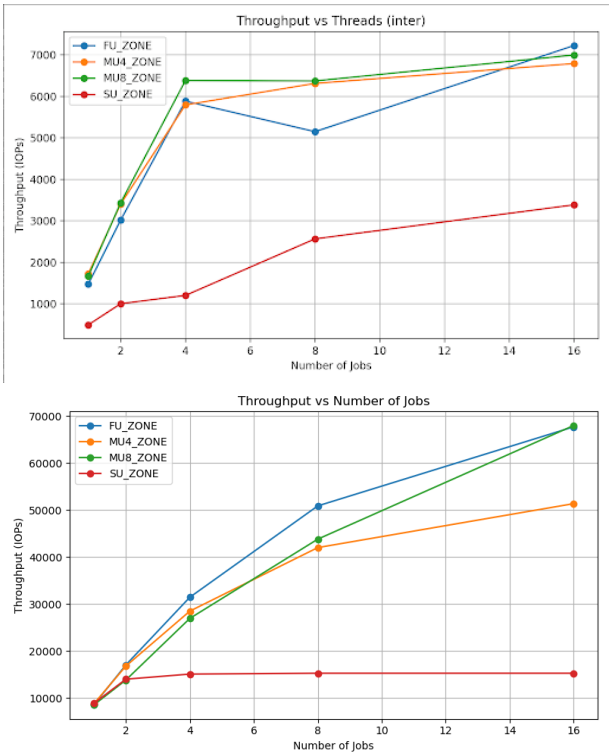

Figure 5: Inter-Zone Throughput; Writes vs Reads

Figure 5 shows the change in Throughput across all configurations as the number of jobs increased. Clearly, throughput increased with the increase in jobs, but the rate of change slowed as the number of jobs hit a threshold. An interesting observation from this is that the SU configuration appears to have a significantly lower throughput than the others. This makes sense, as the SU configuration naturally has lower parallelism compared to the other configurations. This is because only the associated channel

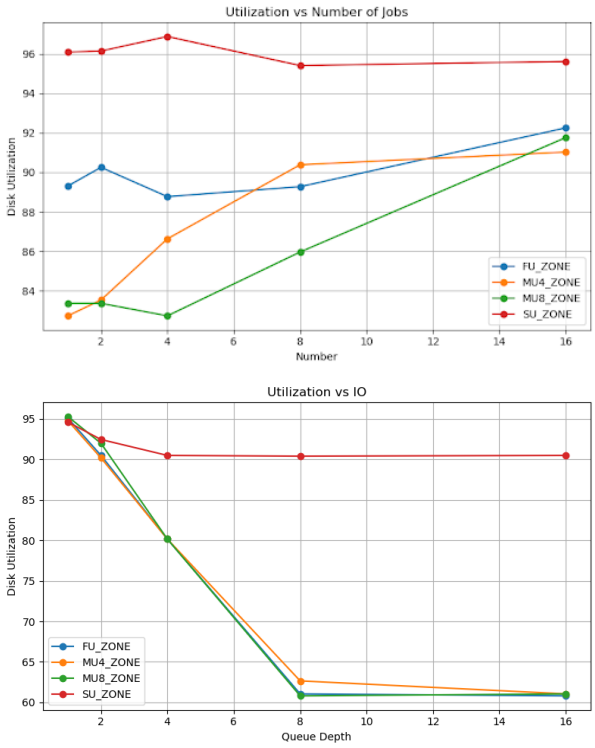

Figure 6: Inter-Zone Utilization; Writes vs Reads

can access their zone, so throughput cannot scale as well, while the other configurations allow for multiple channels to access each zone. Another interesting observation across the tests was seen in figure 6, where the disk utilization is consistently higher for the SU Zone than the others. This is even more pronounced when considering random-reads, when the utilization of SU Zones is stable at around 90 percent, while the other zones drop to 60. This is again because they're less parallel, and each I/O request consumes more exclusive time on its corresponding channel. This is a sign of bottlenecking, not of optimal resource use.

Next, we looked at each configuration individually, comparing their read and write IOPS as the number of jobs increases. This is shown in figures 7-10. Based on these graphs, it's clear that the IOPS operations for reads outpace the operations for writes, which is to be expected. An interesting observation is that for SU, the reads make a massive jump when the number of jobs reaches 40. More testing is probably needed to further ascertain more info from these tests.

4.2 Intra-Zone Parallelism

We also tried running tests to compare Intra-Zone Parallelism. Figure 15 displays the findings. From the graph, we see that while there is a slight uptick in throughput as the queue depth initially increases it remains mostly stable. This is likely because the iodepth parameter is impacted by the sequential write constraints that the ZNS interface provides. A different test would probably be more

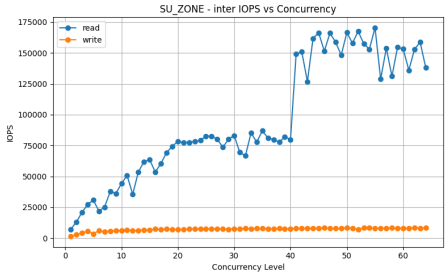

Figure 7: SU-Inter Zone Results

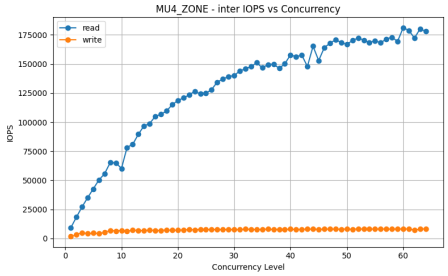

Figure 8: MU^4 Inter Zone Results

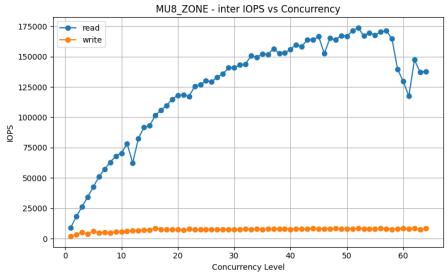

Figure 9: MU^8 Inter Zone Results

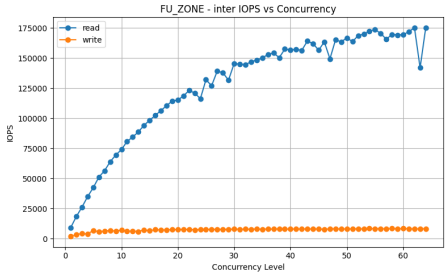

Figure 10: FU Inter Zone Results

suited to check intra-zone parallelism on writes. The graphs for each individual configuration, are shown in figures 11-14.

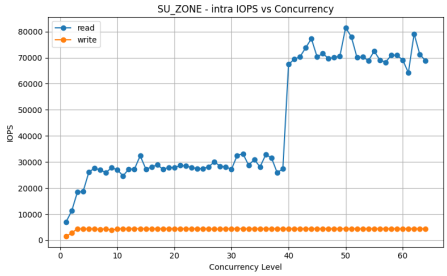

Figure 11: SU-Intra Zone Results

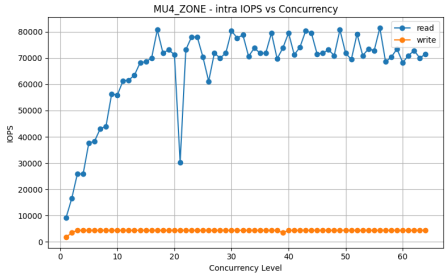

Figure 12: MU^4 Intra Zone Results

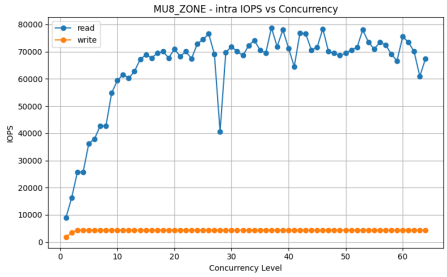

Figure 13: MU^8 Intra Zone Results

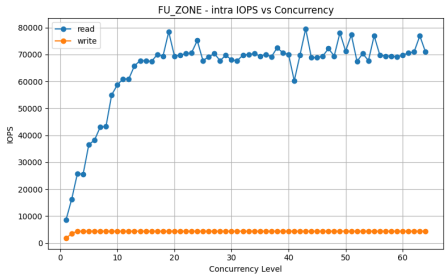

Figure 14: FU Intra Zone Results

4.3 Disk Utilization vs Concurrency

For both of these tests, we also graphed how the disk utilization changes as concurrency increased. These tests were all done using 4kb block sizes, and the results are displayed in figures 16-19. These

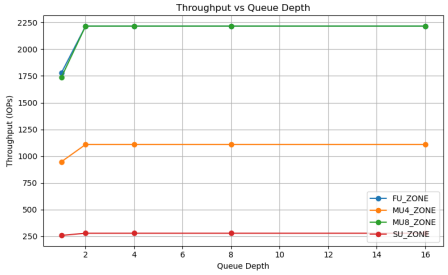

Figure 15: Intra Zone Throughput

graphs seem to indicate that in general the Inter Tests seem to result in the disk utilization increasing, while the Intra Tests show a decrease in disk utilization. Additionally, these also show that there are slight differences between each configuration, even though the results of the throughput tests appear to mirror each other.

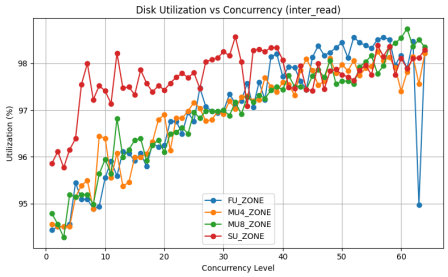

Figure 16: Utilization vs Concurrency Inter Read

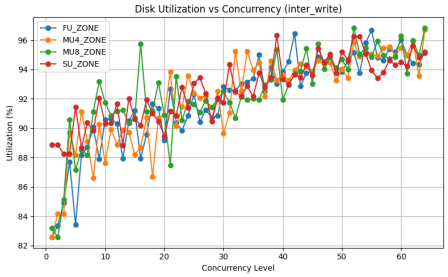

Figure 17: Utilization vs Concurrency Inter Write

5 CONCLUSION

After review of the results, we believe that they are inconclusive, and that more work needs to be done before it is possible to definitively state what the optimal values for concurrency and asymmetry. There are more zone specific operations that can and should be evaluated, like zone-append and zone-finish, which were not covered in these initial experiments. However, we do think that the contents of our repository can be used as a starting framework to conduct these tests.

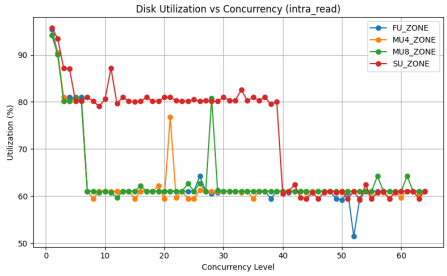

Figure 18: Utilization vs Concurrency Intra Read

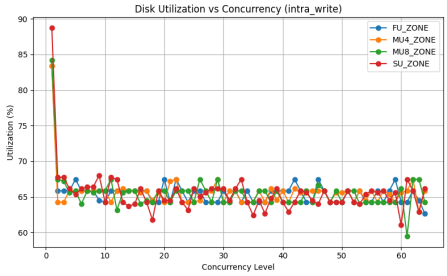

Figure 19: Utilization vs Concurrency Intra Write

REFERENCES

[1] Seiichi Aritome. 2016. NAND Flash Memory Revolution. In *2016 IEEE 8th International Memory Workshop (IMW)*. 1–4. https://doi.org/10.1109/IMW.2016.7495285

[2] Matias Bjørling, Abutalib Aghayev, Hans Holmberg, Aravind Ramesh, Damien Le Moal, Gregory R. Ganger, and George Amvrosiadis. 2021. ZNS: Avoiding the Block Interface Tax for Flash-based SSDs. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, 689–703. https://www.usenix.org/conference/atc21/presentation/bjorling

[3] Li-Pin Chang. 2007. On efficient wear leveling for large-scale flash-memory storage systems. In *Proceedings of the 2007 ACM Symposium on Applied Computing* (Seoul, Korea) *(SAC '07)*. Association for Computing Machinery, New York, NY, USA, 1126–1130. https://doi.org/10.1145/1244002.1244248

[4] Krijn Doekemeijer, Dennis Maisenbacher, Zebin Ren, Nick Tehrany, Matias Bjørling, and Animesh Trivedi. 2024. *Exploring I/O Management Performance in ZNS with ConfZNS++*. Association for Computing Machinery, New York, NY, USA. 162–177 pages. https://doi.org/10.1145/3688351.3689160

[5] David Geer. 2005. Industry Trends: Chip Makers Turn to Multicore Processors . *Computer* 38, 05 (May 2005), 11–13. https://doi.org/10.1109/MC.2005.160

[6] Mingzhe Hao, Gokul Soundararajan, Deepak Kenchammana-Hosekote, Andrew A. Chien, and Haryadi S. Gunawi. 2016. The tail at store: a revelation from millions of hours of disk and SSD deployments. In *Proceedings of the 14th Usenix Conference on File and Storage Technologies* (Santa Clara, CA) *(FAST'16)*. USENIX Association, USA, 263–276.

[7] Minwoo Im, Kyungsu Kang, and Heonyoung Yeom. 2022. Accelerating RocksDB for small-zone ZNS SSDs by parallel I/O mechanism. In *Proceedings of the 23rd International Middleware Conference Industrial Track* (Quebec, Quebec City, Canada) *(Middleware Industrial Track '22)*. Association for Computing Machinery, New York, NY, USA, 15–21. https://doi.org/10.1145/3564695.3564774

[8] Ioannis Koltsidas and Stratis D. Viglas. 2011. Data management over flash memory. In *Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data* (Athens, Greece) *(SIGMOD '11)*. Association for Computing Machinery, New York, NY, USA, 1209–1212. https://doi.org/10.1145/1989323.1989455

[9] Huaicheng Li, Mingzhe Hao, Michael Hao Tong, Swaminathan Sundararaman, Matias Bjørling, and Haryadi S. Gunawi. 2018. The CASE of FEMU: Cheap, Accurate, Scalable and Extensible Flash Emulator. In *16th USENIX Conference on File and Storage Technologies (FAST 18)*. USENIX Association, Oakland, CA, 83–90. https://www.usenix.org/conference/fast18/presentation/li

[10] Tarikul Islam Papon and Manos Athanassoulis. 2021. A Parametric I/O Model for Modern Storage Devices. In *Proceedings of the 17th International Workshop on Data Management on New Hardware* (Virtual Event, China) *(DAMON '21)*. Association for Computing Machinery, New York, NY, USA, Article 2, 11 pages.

https://doi.org/10.1145/3465998.3466003